

Experiment Report

Student: Denisenko Sofiiа 狄苏菲

Student id: 1820249051

Course: Big Data Analysis Technology

Experiment Topic: Text analysis

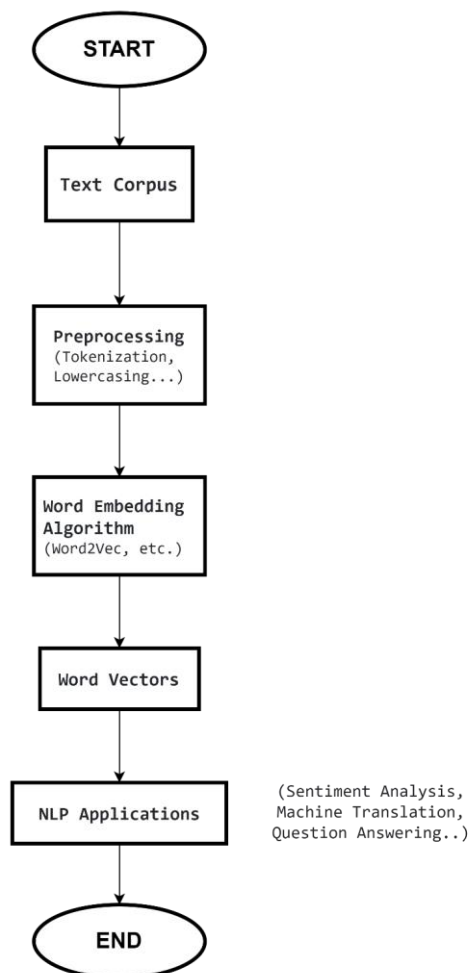
Word Embedding

Word embedding is a technique in natural language processing (NLP) that transforms words into numerical vectors. These vectors capture semantic relationships between words, allowing computers to understand and process language more effectively.

The Essence of Word Embedding:

Imagine a dictionary where each word is represented not by its definition, but by a point in space. Words that are similar in meaning are placed closer together, while dissimilar words are farther apart. This is the core concept of word embedding.

Mechanism and Workflow:



1. Text Corpus:

This is the raw data that will be used to train the embedding algorithm.

2. Preprocessing:

This stage prepares the text for the embedding process.

- **Tokenization:** The text is broken down into individual words, like separating “The quick brown fox” into “The”, “quick”, “brown”, “fox”.
- **Lowercasing:** All words are converted to lowercase, so “The” and “the” are treated as the same word.
- **Removing Stop Words:** Common words that don’t carry much meaning (like “the”, “a”, “is”) are removed.
- **Lemmatization/Stemming:** Words are reduced to their base form. For example, “running” becomes “run” or “cats” becomes “cat”.

3. Word Embedding Algorithm:

It’s an algorithm that finds the relationships between words from the preprocessed text. Most popular algorithms are:

- **Word2Vec:** Learns by predicting surrounding words in a sentence.
- **GloVe:** Uses global co-occurrence statistics (how often words appear together).
- **FastText:** Similar to Word2Vec but considers character-level information, making it better for rare words.

4. Word Vectors:

The output of the algorithm. Each unique word from the original text corpus is now represented by a numerical vector. These vectors have a specific number of dimensions. The key point is that **words with similar meanings are placed closer together in this vector space**, while dissimilar words are farther apart.

5. NLP Applications:

The vectors are now used to improve various NLP tasks:

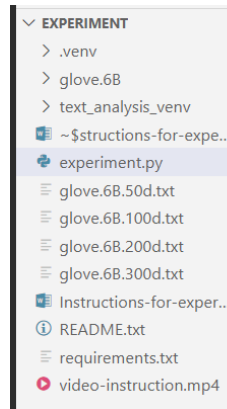
- **Sentiment Analysis:** Classifying text as positive, negative, or neutral.
- **Machine Translation:** Translating text from one language to another.
- **Text Classification:** Categorizing text into different topics.
- **Question Answering:** Answering questions based on given text.
- **Speech Recognition:** Converting speech to text more accurately.

Experiment

At first, I started a new project with virtual environment using VS Code, and downloaded dependencies.

```
PS D:\Китай\Учѐба\Big Data Analysis\Text analysis\experiment> pip install tensorflow pandas numpy matplotlib nltk scikit-learn
Requirement already satisfied: tensorflow in d:\китай\учѐба\big data analysis\text analysis\experiment\.venv\lib\site-packages (2.17.0)
Collecting pandas
  Obtaining dependency information for pandas from https://files.pythonhosted.org/packages/ed/8c/87ddf1fcb55d11f9f847e3c69bb1c6f8e46e2f40ab1a2d2abadb2401b007/pa
das-2.2.3-cp311-cp311-win_amd64.whl.metadata
  Using cached pandas-2.2.3-cp311-cp311-win_amd64.whl.metadata (19 kB)
Requirement already satisfied: numpy in d:\китай\учѐба\big data analysis\text analysis\experiment\.venv\lib\site-packages (1.26.4)
Collecting matplotlib
  Obtaining dependency information for matplotlib from https://files.pythonhosted.org/packages/8b/ce/15b0bb2fb29b3d46211d8ca740b96b5232499fc49200b58b8d571292c9a
/matplotlib-3.9.2-cp311-cp311-win_amd64.whl.metadata
  Using cached matplotlib-3.9.2-cp311-cp311-win_amd64.whl.metadata (11 kB)
Collecting nltk
```

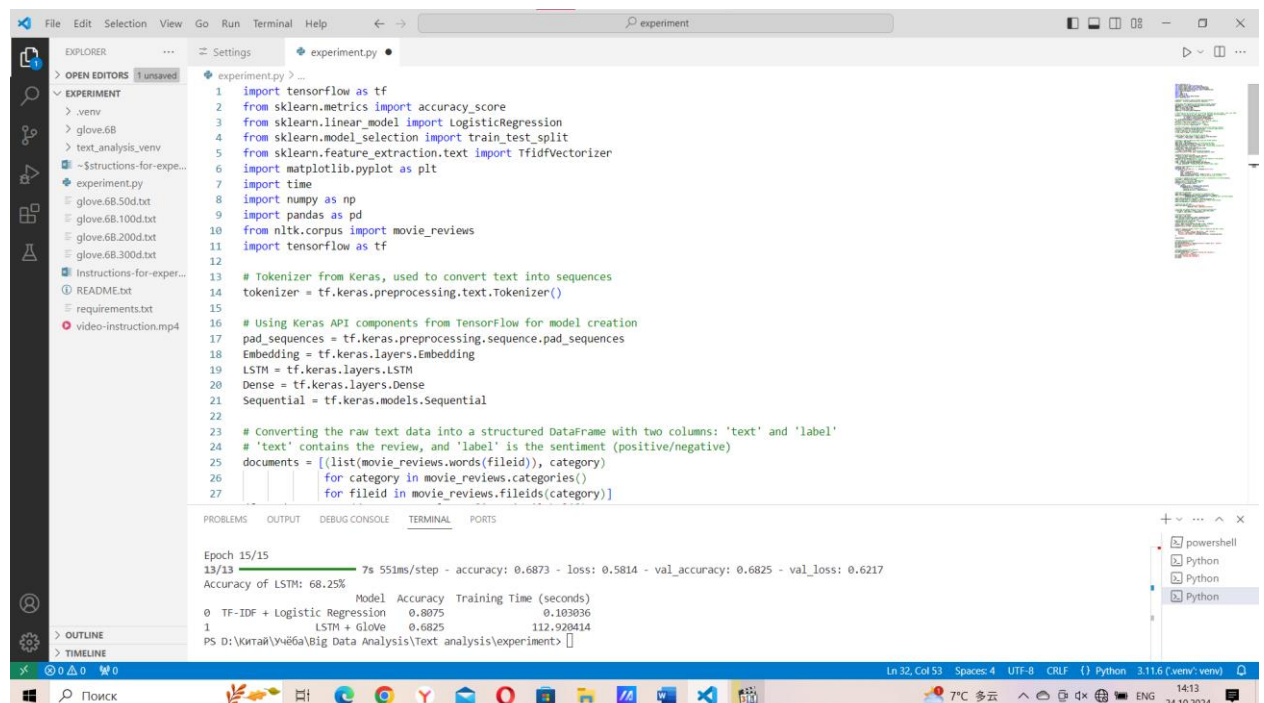
Installing GloVe6B and unpacking it in the folder.



Downloading movie_reviews dataset.

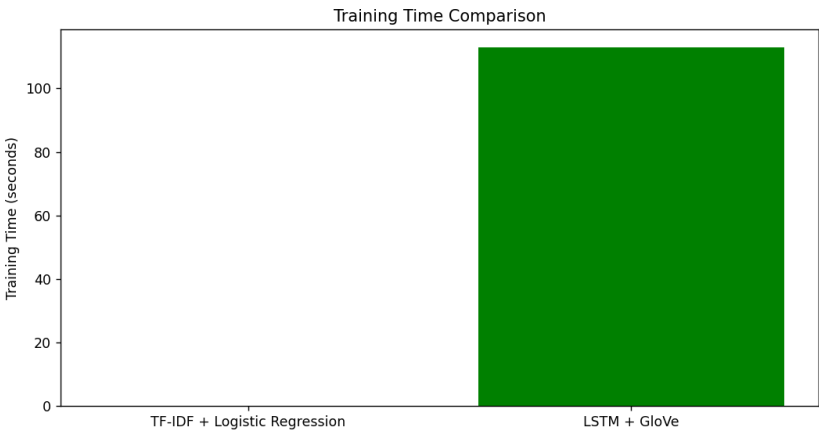
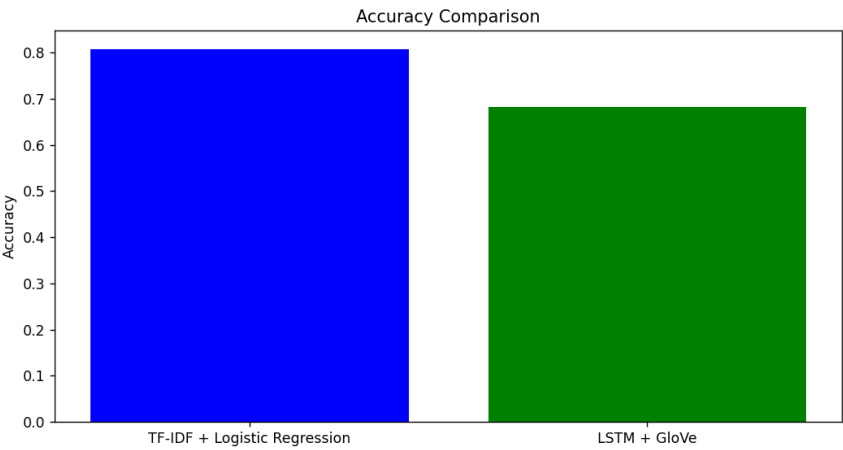
```
PS D:\Китай\Учѐба\Big Data Analysis\Text analysis\experiment> python -m nltk.downloader movie_reviews
><frozen runpy>:128: RuntimeWarning: 'nltk.downloader' found in sys.modules after import of package 'nltk', but prior to execution of 'nltk.downloader'; this may
result in unpredictable behaviour
[nltk_data] Downloading package movie_reviews to
[nltk_data] C:\Users\prett\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\movie_reviews.zip.
```

Pasting and running the code.



The results are showed below.

```
2024-10-24 14:05:51.238236: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating
-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2024-10-24 14:05:52.375525: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating
-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Accuracy of TF-IDF model: 80.75%
2024-10-24 14:06:11.326893: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in perform
ance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Epoch 1/15
13/13 ----- 12s 570ms/step - accuracy: 0.5190 - loss: 0.7008 - val_accuracy: 0.5475 - val_loss: 0.6865
Epoch 2/15
13/13 ----- 7s 510ms/step - accuracy: 0.5707 - loss: 0.6806 - val_accuracy: 0.5375 - val_loss: 0.6867
Epoch 3/15
13/13 ----- 7s 515ms/step - accuracy: 0.5842 - loss: 0.6681 - val_accuracy: 0.5575 - val_loss: 0.6839
Epoch 4/15
13/13 ----- 7s 522ms/step - accuracy: 0.6254 - loss: 0.6539 - val_accuracy: 0.5825 - val_loss: 0.6719
Epoch 5/15
13/13 ----- 7s 516ms/step - accuracy: 0.6346 - loss: 0.6437 - val_accuracy: 0.5675 - val_loss: 0.6863
Epoch 6/15
13/13 ----- 7s 534ms/step - accuracy: 0.6187 - loss: 0.6398 - val_accuracy: 0.5700 - val_loss: 0.6740
Epoch 7/15
13/13 ----- 7s 549ms/step - accuracy: 0.6376 - loss: 0.6385 - val_accuracy: 0.6150 - val_loss: 0.6602
Epoch 8/15
13/13 ----- 7s 541ms/step - accuracy: 0.6507 - loss: 0.6263 - val_accuracy: 0.6300 - val_loss: 0.6479
Epoch 9/15
13/13 ----- 9s 689ms/step - accuracy: 0.6632 - loss: 0.6080 - val_accuracy: 0.6625 - val_loss: 0.6402
Epoch 10/15
13/13 ----- 7s 538ms/step - accuracy: 0.6880 - loss: 0.6010 - val_accuracy: 0.6350 - val_loss: 0.6505
Epoch 11/15
13/13 ----- 7s 529ms/step - accuracy: 0.6831 - loss: 0.5939 - val_accuracy: 0.6750 - val_loss: 0.6327
Epoch 12/15
13/13 ----- 7s 545ms/step - accuracy: 0.6682 - loss: 0.6032 - val_accuracy: 0.6875 - val_loss: 0.6207
Epoch 13/15
13/13 ----- 8s 649ms/step - accuracy: 0.6782 - loss: 0.5872 - val_accuracy: 0.6850 - val_loss: 0.6114
Epoch 14/15
13/13 ----- 8s 551ms/step - accuracy: 0.6843 - loss: 0.5896 - val_accuracy: 0.6425 - val_loss: 0.6295
Epoch 15/15
13/13 ----- 7s 551ms/step - accuracy: 0.6873 - loss: 0.5814 - val_accuracy: 0.6825 - val_loss: 0.6217
Accuracy of LSTM: 68.25%
Model Accuracy Training Time (seconds)
0 TF-IDF + Logistic Regression 0.8075 0.103036
1 LSTM + GloVe 0.6825 112.920414
PS D:\Китай\Учёба\Big Data Analysis\Text analysis\experiment> █
```



Analysis: As we can see the performance of TF-IDF + Logistic Regression is better than LSTM + GloVe model. The accuracy is slightly better, the training time is very much better.

Changing the number of epochs from 15 to 3.

```
# Training the LSTM model
start_time = time.time()
model_lstm.fit(X_train_pad, y_train, validation_data=(
    X_test_pad, y_test), epochs=3, batch_size=128)
# Calculating training time for LSTM
training_time_lstm = time.time() - start_time
# Evaluating the LSTM model
```

PS D:\Китай\Учёба\Big Data Analysis\Text analysis\experiment> & "d:/Китай/Учёба/Big Data Analysis/Text analysis/experiment/.venv/Scripts/python.exe" "d:/Китай/Учёба/Big Data Analysis/Text analysis/experiment/experiment.py"

2024-10-24 14:25:15.277386: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.

2024-10-24 14:25:16.391569: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.

Accuracy of TF-IDF model: 80.75%

2024-10-24 14:25:33.548112: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

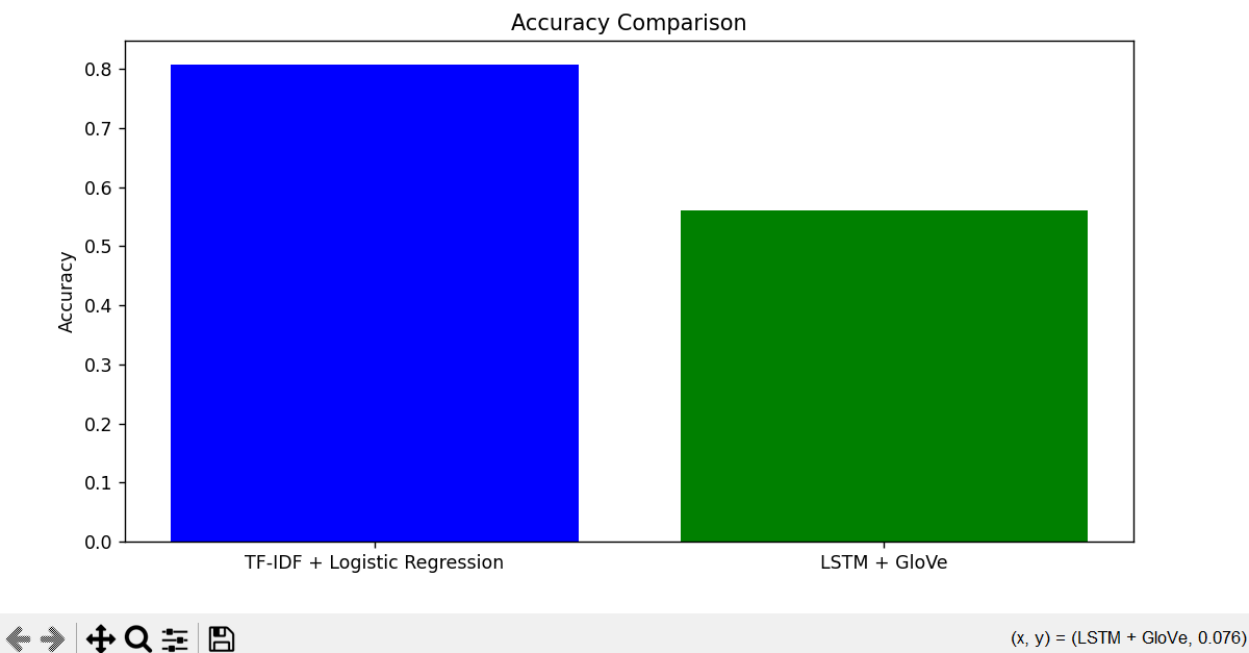
Epoch 1/3
13/13 9s 548ms/step - accuracy: 0.5166 - loss: 0.6988 - val_accuracy: 0.5575 - val_loss: 0.6863

Epoch 2/3
13/13 7s 509ms/step - accuracy: 0.5574 - loss: 0.6794 - val_accuracy: 0.5700 - val_loss: 0.6817

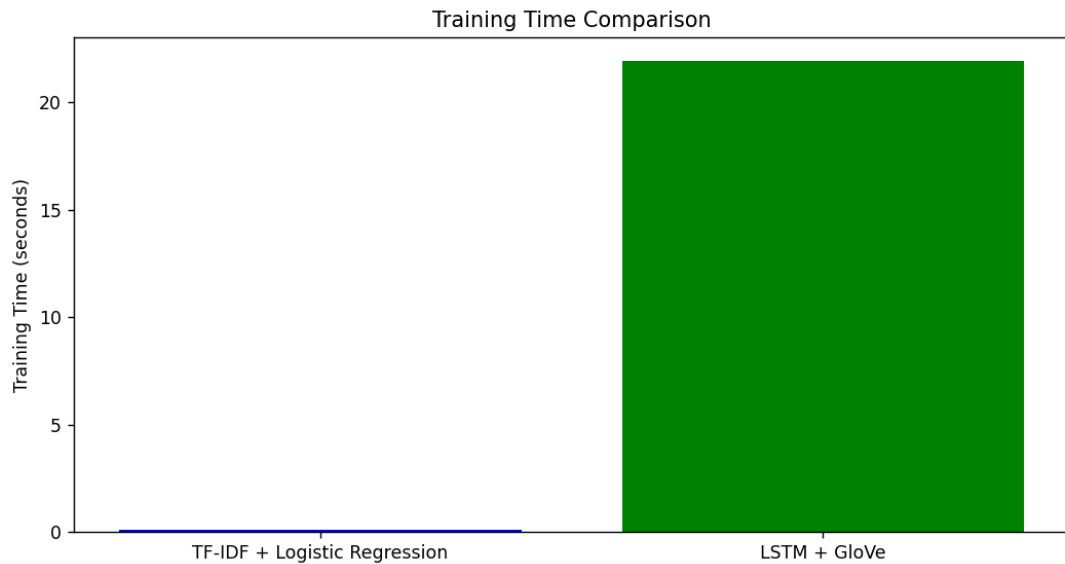
Epoch 3/3
13/13 7s 508ms/step - accuracy: 0.5876 - loss: 0.6659 - val_accuracy: 0.5600 - val_loss: 0.6848

Accuracy of LSTM: 56.00%

	Model	Accuracy	Training Time (seconds)
0	TF-IDF + Logistic Regression	0.8075	0.090031
1	LSTM + GloVe	0.5600	21.931440



(x, y) = (LSTM + GloVe, 0.076)



Analysis: By changing the number of epochs, the results in comparison stayed almost the same.

Changing the number of epochs to 30.

```

97 # Splitting the padded sequences into training and testing sets
98 X_train_pad, X_test_pad, y_train, y_test = train_test_split(
99     X_pad, y, test_size=0.2, random_state=42)
100
101 # Training the LSTM model
102 start_time = time.time()
103 model_lstm.fit(X_train_pad, y_train, validation_data=(
104     X_test_pad, y_test), epochs=30, batch_size=128)
105 # Calculating training time for LSTM
106 training_time_lstm = time.time() - start_time
107 # Evaluating the LSTM model
108 scores = model_lstm.evaluate(X_test_pad, y_test, verbose=0)
109 accuracy_lstm = scores[1] # Extracting accuracy
110 print(f"Accuracy of LSTM: {accuracy_lstm*100:.2f}%")
111
112 # Results comparison between TF-IDF + Logistic Regression and LSTM + GloVe

```

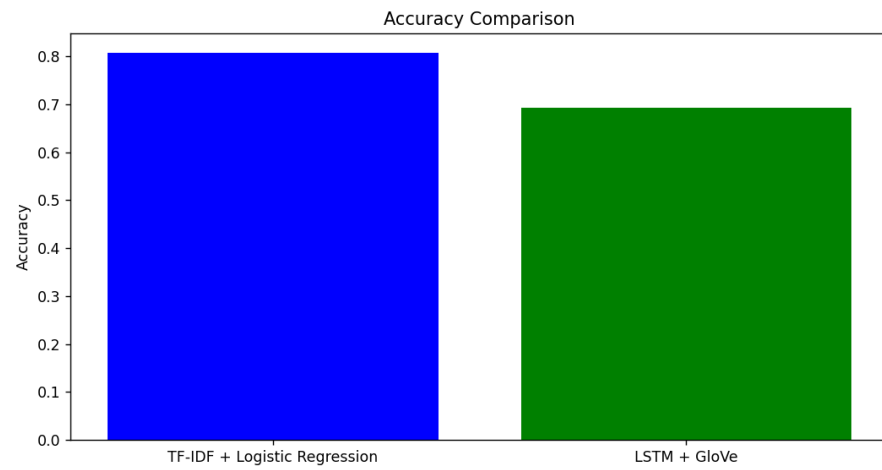
```

Epoch 23/30
13/13 ----- 7s 533ms/step - accuracy: 0.7487 - loss: 0.5060 - val_accuracy: 0.7100 - val_loss: 0.5922
Epoch 24/30
13/13 ----- 7s 539ms/step - accuracy: 0.7557 - loss: 0.4970 - val_accuracy: 0.6875 - val_loss: 0.5971
Epoch 25/30
13/13 ----- 7s 534ms/step - accuracy: 0.7557 - loss: 0.5020 - val_accuracy: 0.6900 - val_loss: 0.6110
Epoch 26/30
13/13 ----- 7s 551ms/step - accuracy: 0.7716 - loss: 0.4824 - val_accuracy: 0.6900 - val_loss: 0.6074
Epoch 27/30
13/13 ----- 7s 533ms/step - accuracy: 0.7660 - loss: 0.4856 - val_accuracy: 0.6100 - val_loss: 0.6757
Epoch 28/30
13/13 ----- 7s 543ms/step - accuracy: 0.7571 - loss: 0.5101 - val_accuracy: 0.6775 - val_loss: 0.6309
Epoch 29/30
13/13 ----- 7s 542ms/step - accuracy: 0.7819 - loss: 0.4632 - val_accuracy: 0.7075 - val_loss: 0.5972
Epoch 30/30
13/13 ----- 7s 559ms/step - accuracy: 0.7917 - loss: 0.4433 - val_accuracy: 0.6925 - val_loss: 0.6021
Accuracy of LSTM: 69.25%

```

	Model	Accuracy	Training Time (seconds)
0	TF-IDF + Logistic Regression	0.8075	0.090092
1	LSTM + GloVe	0.6925	21.344097

PS D:\Корна\Учеба\Big Data Analysis\Text analysis\experiment>



As we can see, results stayed the same. TF-IDF + Logistic Regression is better in everything.